Project 2 Plan: Mobile Money Investigations (PostgreSQL and Neo4j)

Oct 19, 2026 · @hello claude connect to my powerbi project called cenfri pi on

The project is due 26 October 2026 at 11:59 PM CAT, so about 7 of the suggested 14 days remain. Loading both databases must finish within the first 2 days, because every query, benchmark and decision section depends on it.

The plan follows the handout in milestone order: setup, capture, PostgreSQL, Neo4j, queries Q1 to Q7, correctness and timing, concurrency, decisions and submission. Marks per phase are shown in each heading (100 in total).

Phase 0: Setup (Section 1.1, 2 marks)

1. Install or confirm PostgreSQL, Neo4j Community, Python 3, `psycopg`, the Neo4j Python driver, `numpy` and `matplotlib`. Check that about 10 GB of disk is free.
2. Create the project PostgreSQL database and set its time zone to Africa/Kigali (`ALTER DATABASE ... SET timezone = 'Africa/Kigali'`).
3. Create a dedicated Neo4j instance that uses the default `neo4j` database.
4. Record software and driver versions, CPU, RAM, OS and memory settings (`shared_buffers`, `work_mem`, Neo4j heap and page cache). Screenshot them with the taskbar and clock visible.
5. Put `mobile_money_generator.py` under version control now and record its hash, since Section 9.2 asks you to prove it is unchanged.

Phase 1: Capture (Section 1.2, 5 marks)

Write `capture.py` so that it streams all five generator outputs to CSV without ever holding a full stream in memory.

1. `import mobile_money_generator as g`, then print `g.dataset_info()` and `g.FIELDS`.
2. Preview 10 rows from each stream with `itertools.islice`.
3. For each stream, write the header first, then rows as they arrive, using `csv.DictWriter`. Count rows consumed and rows written.
4. For transfers, also accumulate the row count, earliest and latest `occurred_at`, and the sum of `amount_rwf`. Log every 1,000,000 rows and record capture seconds and CSV sizes.

5. Check the totals: 228,000 customers, 248,000 accounts, 228,600 devices, 285,600 account_devices and 20,003,300 transfers (20,993,500 records in all).
6. Save the audit totals to a JSON file, because both database loads are checked against it.

Phase 2: PostgreSQL (Sections 2.1 and 2.2, 9 marks)

1. **Schema.** Create five tables with every column `NOT NULL`. Use `INTEGER` for entity IDs, amounts and observations, `BIGINT` for `transfer_id` and `occurred_at`, `SMALLINT` for observation days and `TEXT` for categories.
2. **Keys and checks.** Add primary keys, with `(account_id, device_id)` as the key of `account_devices`. Add foreign keys for ownership, device use and both transfer endpoints. Add checks for `amount_rwf > 0`, `observations > 0` and `0 <= first_day <= last_day <= 29`. Draw the schema and explain each field and unit.
3. **Load.** Use `\copy` in dependency order: customers, devices, accounts, account_devices, transfers. Time loading separately from index and constraint setup. Loading transfers before adding foreign keys and indexes is much faster at 20M rows.
4. **Indexes.**
   - `accounts(customer_id, account_id)`
   - `account_devices(device_id, account_id)`
   - `transfers(sender_id, occurred_at)` INCLUDE `receiver_id, amount_rwf, transfer_id`
   - `transfers(receiver_id, occurred_at)` INCLUDE `sender_id, amount_rwf, transfer_id`
   - a BRIN index on `transfers(occurred_at)`, then `ANALYZE`
5. **Verify.** Match every row count and the transfer audit totals to `capture.py`. Confirm 240,000 person, 6,000 merchant and 2,000 agent accounts. Show validated constraints, indexes and sizes, using 1 GB = 1,000,000,000 bytes.

Phase 3: Neo4j (Sections 3.1 to 3.3, 12 marks)

1. **Model.** Nodes are `Customer`, `Account` and `Device`. Relationships are `OWNS`, `USED_DEVICE` (keeping `first_day`, `last_day`, `observations`) and `TRANSFER` (keeping `transfer_id`, `occurred_at`, `amount_rwf`). Every transfer is its own relationship, never collapsed. Draw the graph schema and map each source field to it.
2. **`prepare_import.py`.** Make temporary copies of the CSVs with typed headers and leave the source CSVs unchanged. Use `customer_id:ID(Customer)`, `account_id:ID(Account)` and `device_id:ID(Device)`. The accounts header is

`account_id:ID(Account),:IGNORE,role:string`. Derive `owns.csv` from accounts in customer_id, account_id order. Use `:int` or `:long` for numeric properties.

3. **Import.** Stop Neo4j, then run `neo4j-admin database import full neo4j --id-type=integer` with `--nodes=Label=file.csv` for all three node files and `--relationships=TYPE=file.csv` for all three relationship files. Record import seconds and store and transaction-log sizes.
4. **After import.** Start Neo4j, create unique constraints on the three business IDs and a range index on `TRANSFER.occurred_at`, and wait until the indexes are online.
5. **Verify.** Expect 704,600 nodes and 20,536,900 relationships in total: OWNS 248,000, USED_DEVICE 285,600, TRANSFER 20,003,300. Also check role counts, one owner per account, unique `transfer_id` and the capture audit totals. Then delete the temporary import copies.
6. **Trace one account in both models (3.3).** Use SQL JOINs and Cypher `MATCH` from one account to its owner, a device and an outgoing transfer. Explain how foreign keys become relationships, and what time and event evidence would be lost if repeated payments were collapsed into one edge.

Phase 4: Query pairs Q1 to Q7 (Sections 4 and 5, 40 marks)

Build one parameterized SQL and Cypher pair per question and run it on all six sets A to F.

Time conversion. The data starts at `START = 1788213600`, which is 1 September 2026 00:00 CAT. For September day `d`, the hour window is `START + (d-1)*86400 + 9*3600 + 45*60` (inclusive) to that value plus 3600 (exclusive). Compute it in Python and pass it as `%(start)s` and `%(end)s` in SQL, or `$start` and `$end` in Cypher.

Question	Core idea	Pitfalls
Q1: account activity (6 cases)	Outgoing count and total RWF in the hour, all receiver roles	Return zeros when nothing matches: `COALESCE` in SQL, and an aggregate in Cypher that still yields a row. Explain why outgoing value is not a balance.
Q2: daily activity (1 case)	Group the full month by CAT day: date, count, total RWF	Use `to_timestamp(...) AT TIME ZONE 'Africa/Kigali'` and `datetime({epochSeconds: ..., timezone: 'Africa/Kigali'})`. Expect 30 rows that reconcile to the capture totals. Plot count and value with units.

Question	Core idea	Pitfalls
Q3: shared devices (6 cases)	Accounts on the same device with overlapping day intervals	Overlap rule: `GREATEST(first_days) <= LEAST(last_days)`. Exclude the review account, include both device types, return overlap start and end, sort by account then device.
Q4: ownership and device network (24 cases)	Accounts reachable through OWNS or USED_DEVICE in either direction within 4, 6, 8 and 10 links, grouped by owner region	In SQL carry node kind plus ID, since Customer 7 is not Account 7, and use `UNION` so the recursion terminates. In Cypher use an undirected bounded path, then `DISTINCT` accounts. Write predictions down before running depths 8 and 10.
Q5: rapid onward payments (6 cases)	Three directed transfers through four distinct accounts, the first by the review account in its hour	Each next transfer is strictly later and at most 600 seconds later. Require `90*prev <= 100*next <= 105*prev` with BIGINT arithmetic.
Q6: circular payments (6 cases)	3 to 8 transfers that start and end at the review account, all inside the hour and spanning at most 1,800 seconds	Recursive SQL carrying current account, ordered transfer IDs, first and last time, and last amount. Stop at the first return. No 600-second gap rule applies.
Q7: shortest transfer connection (12 cases)	Fewest directed transfers to the destination, bounds 4 and 6, every account on the route a person	Apply the person restriction during the search, not afterwards. In Cypher use `shortestPath` with an `all(n IN nodes(p) WHERE n.role = 'person')` predicate. Retrieve and verify one path per successful case separately.

For every question, screenshot the query pair, a summary table of all cases and one worked example, and write the short explanations the handout asks for.

Phase 5: Correctness and performance (Section 6, 14 marks)

1. **Comparison program (6.1).** Write `run_pg(query, params)` and `run_neo(query, params)`, each returning every row. Normalize integers and sort before exact comparison. Cover 61 cases (6 + 1 + 6 + 24 + 6 + 6 + 12). Compare Q7 lengths separately from valid supporting paths. Independently verify one device overlap and one chain or cycle against the CSV rows in plain Python.
2. **Timing (6.2).** Time 21 cases per engine: Q1, Q3, Q5 and Q6 on B and E; Q2 for the full month; Q4 on B and E at all four depths; Q7 on B and E at both bounds. Use persistent connections, one database running at a time and fixed memory settings. Run 3 untimed warm-ups (180 s limit) then 5 measured runs (60 s limit) per case, timed with `perf_counter` through the last row. Log errors and timeouts separately, never as timings.
3. **Reporting.** Save case, parameters, engine, CAT start time, duration, row count and status. Report median, minimum and maximum milliseconds and the PostgreSQL/Neo4j median ratio.
4. **Plans.** Outside the timer, collect `EXPLAIN (ANALYZE, BUFFERS)` and `PROFILE` for Q2, Q4 B depth 10 and Q7 B bound 6, and explain one processing difference.
5. **SQL improvement (6.3).** Use the Q7 plan to find wasted search work, then try one change: deduplicate account-depth states, search from both ends, or stop at a proven minimum. Keep both versions, recheck all 12 cases, retime B and E at both bounds, and pick the version for later sections by bound-6 medians. A justified unsuccessful attempt can earn full marks.

Phase 6: Concurrency (Section 7, 9 marks)

1. **Design.** Use Python `multiprocessing` with one persistent connection per process and a `Barrier`. The workloads are Q4 depth 10 and Q7 length-only bound 6 (using your selected SQL version), cycling A, B, C. Test 1, 4, 8 and 16 clients on each engine: 2 engines × 2 workloads × 4 client counts = 16 combinations, each repeated 3 times.
2. **Run.** Each client warms up A-B-C once. At barrier release set a common start time and a deadline 15 seconds later. Keep starting cycles until the deadline, finish any cycle already started, and submit each next request immediately. Keep the 60-second query timeout.
3. **Measure.** Log every request and check correctness after timing. Throughput is correct completions divided by the time from barrier release to the last cycle finishing. Report throughput, median latency, p95 (sorted successful latencies at position `ceil(0.95*n)`), errors and timeouts per repetition, then take the median across repetitions.

4. **Explain (7.2).** Plot throughput and p95 against client count for both workloads and engines. Compare 1 against 16 clients, explain any saturation with CPU, RAM and plan evidence, and keep throughput distinct from individual response time. State whether any graph advantage survives your SQL improvement, limiting claims to this experiment.

Phase 7: Decisions and writing (Section 8, 13 marks)

- **8.1 Prioritization.** Build one table with three rows (A/D, B/E, C/F) using Q3 to Q7, and choose which account to review first or justify equal priority. Discuss one pair in detail with IDs, amounts, times and device evidence. Include one transfer chain or cycle diagram and one ownership-device diagram. Give an ordinary explanation for one suspicious association, name the missing evidence and say how it could change your judgement. Do not declare fraud.
- **8.2 Recommendation (500 words at most).** Assign reporting and investigation work to PostgreSQL, Neo4j or both, using three comparisons (reporting, traversal, concurrency) plus load time and storage. Explain one SQL/Cypher and plan comparison, include evidence against your preferred choice, and separate graph-model benefits from implementation effects. If operational data is copied into a graph, propose a freshness target and the consequences of stale data. State two limits of the recommendation.
- **8.3 Individual notes.** Each member writes 150 to 200 named words on a different measured finding, its explanation and a check that could overturn it. List contributions and any LLM or external assistance, with how it was checked.

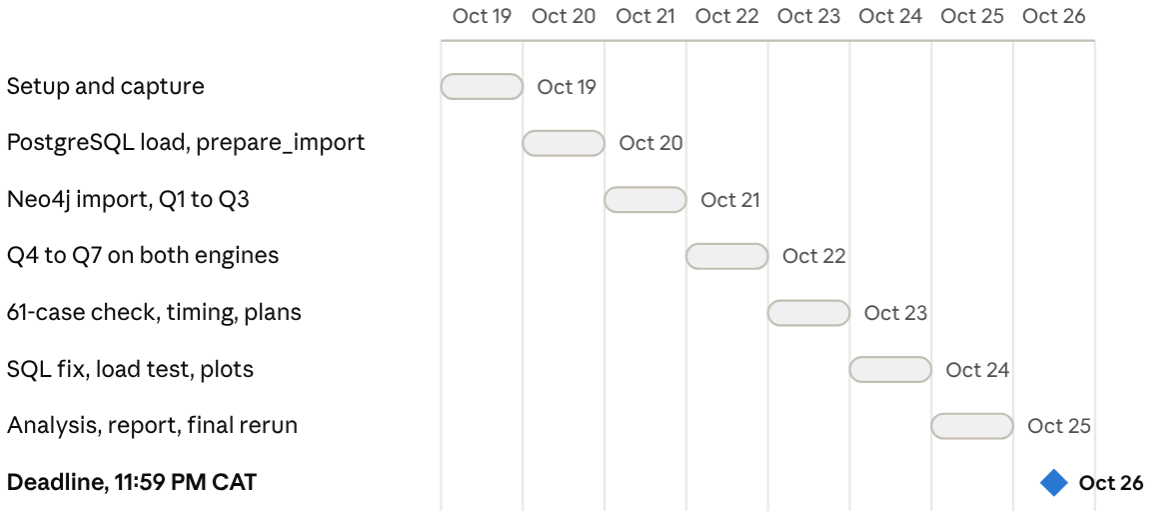
Phase 8: Submission (Section 9, 4 marks)

1. **Report.** One PDF in milestone order with names and IDs, schemas, summary tables, charts, embedded screenshots (taskbar and clock visible) and individual notes. Reference earlier screenshots instead of repeating them, and keep complete outputs and raw repetitions in the code submission.
2. **Code.** A GitHub URL and a matching ZIP containing the unchanged generator, `capture.py`, `prepare_import.py`, definitions, queries, checks, benchmarks, raw results and plans. The README lists commands in execution order with placeholder secrets. Do not commit the CSVs or database stores.
3. **Final checks.** Rerun the 61-case comparison and screenshot its completion. Stop both database services and screenshot the stopped status. Keep your stores.

Suggested calendar for the 7 days left

Both databases are loaded by Oct 21; Oct 25 is kept for writing

Day-by-day plan, 19 to 26 October 2026 (CAT)



7-day plan · 8 rows, deadline 26 Oct

The daily slots are a guide: the 20M-row capture and both loads are the steps most likely to overrun, so start them first.

Things to watch

- **Start the 20M-row capture and load early.** They are the slowest steps and block everything else.
- **Screenshot as you go.** The taskbar and clock requirement means evidence cannot easily be recreated later.
- **Distinguish IDs by kind.** Q4's recursive SQL must match both node kind and ID.
- **Apply Q7's person restriction during the search**, not after choosing a shortest path.
- **Never hard-code answers.** Everything must come from the stored data, not from generator internals.
- **Keep the generator unchanged**, and keep CSVs and database stores out of the repository.